REPALOGIC GROUP — SENIOR PHP BACKEND ENGINEERING ASSESSMENT

Technical Architecture & Implementation Dossier

Candidate: Megharaj P N

Runtime: Native PHP 8.1+ (Strict Types)

Repository: MegharajPN/repalogic-technical-assessment

0% Frameworks	2 Core Engines	10 / 10 Passed	In-Memory State
Pure Native PHP 8.1+	Policy & Settlement	Strict CLI Assertions	Atomic Rollback Safe

01 Executive Architectural Overview

This technical dossier documents the design, domain modeling, and verification strategy for the Repalogic Senior Backend Engineer assessment. The project addresses two critical Enterprise Application Integration (EAI) subsystems: a policy evaluation gatekeeper (`TenantAccessGuard`) for high-resource background automation tasks, and a high-velocity B2B transactional processing engine (`SettlementEngine`) featuring multi-currency conversion, rolling-window overdraft velocity protection, vendor circuit breaking, and atomic batch rollback.

Every class is built with `declare(strict_types=1);`, encapsulated domain state, immutable data transfer objects (`readonly` properties), and zero third-party packages.

02 Challenge 1: Core Logic & Policy Engine (TenantAccessGuard)

Located in `src/Policy/TenantAccessGuard.php`, the policy engine evaluates tenant eligibility using deterministic guard clauses ordered by business severity. By prioritizing account suspension checks first, followed by tier-specific overdue constraints and dynamic buffer thresholds (`hardCap × 1.10`), the engine eliminates deeply nested branching and guarantees `O(1)` evaluation performance.

TEST ID	SUBSCRIPTION TIER	USAGE / CAP	INVOICE STATUS	ENGINE OUTPUT	ARCHITECTURAL RULE VERIFIED
TC-01	Standard	800 / 1000	Active	`ALLOWED`	Baseline happy path under standard hard cap.
TC-02	Standard	500 / 1000	Overdue	`BLOCKED_STATUS`	Standard tier has zero overdue grace period even under cap.
TC-03	Standard	1050 / 1000	Active	`BLOCKED_USAGE`	Strict hard cap enforcement on active standard accounts.
TC-04	Premium	1080 / 1000	Overdue	`ALLOWED`	Grace Period rule allows up to +10% buffer (1100 ceiling).
TC-05	Enterprise	1150 / 1000	Overdue	`BLOCKED_USAGE`	Exceeding the +10% grace ceiling (1150 > 1100) blocks task.
TC-06	Enterprise	100 / 1000	Suspended	`BLOCKED_STATUS`	Suspended status immediately blocks execution across all tiers.

03 Challenge 2: B2B Analytical Settlement Engine Pipeline

Located in `src/Settlement/`, the `SettlementEngine` processes sequential and batch transaction streams from external API vendors. Every incoming `Transaction` passes through a strict four-stage execution pipeline before mutating account state or appending to the internal audit ledger.

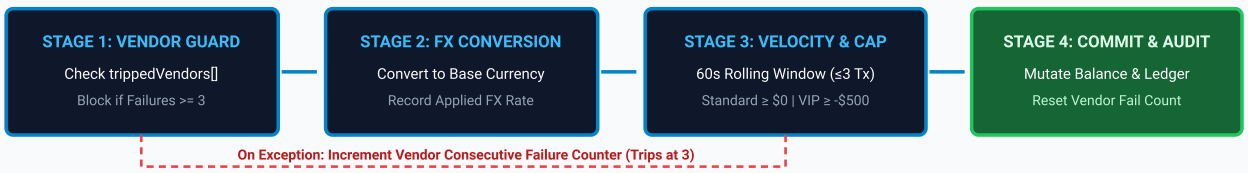


Figure 1: Deterministic execution lifecycle inside SettlementEngine::processTransaction()

04 Core Algorithmic Implementations

- **Multi-Currency Conversion & Audit Precision:** Incoming transactions are normalized to the target account's base currency via cross-rate calculation (`rateFromUsdToSource / rateFromUsdToBase`). Every successful transaction records `original_amount`, `rate_applied`, `base_impact`, and `new_balance` inside `$auditLedger`.
- **Rolling 60-Second VIP Velocity Trap:** VIP accounts receive a hard overdraft protection floor of `-$500.00 USD`. However, `Account::getDeductionCountInWindow()` inspects all deduction timestamps within `[currentTimestampMs - 60000, currentTimestampMs]`. When a 4th deduction is attempted within 60 seconds (`priorDeductions + 1 > 3`), `revokeOverdraft()` is triggered immediately and sub-zero balances throw `InsufficientFundsException`.
- **Cascading Vendor Fraud Circuit Breaker:** Each exception thrown during transaction processing increments `$vendorFailureCounts[$vendorId]`. A valid transaction resets the vendor's counter to `0`. Upon reaching 3 consecutive failures, `$trippedVendors[$vendorId]` locks the vendor, immediately throwing and logging `CircuitBreakerException` on all subsequent requests before account lookup.

> **Key Engineering Highlight: In-Place Atomic Batch Rollback (Memento Pattern)**
>
> In PHP, using `clone $account` during batch snapshotting creates a new object reference in memory. If a batch rolls back by replacing `$this->accounts[$id]` with the cloned reference, any external caller holding a pointer to the original `$account` variable sees stale mutated state. To guarantee 100% reference integrity, `Account::createSnapshot()` and `Account::restoreSnapshot()` capture and restore state **in-place on the exact same object instance** when `processBatch()` catches an exception.

05 Settlement Engine Verification Matrix (Scenarios 1–4)

The CLI runner (`run_tests.php`) enforces strict assertions (`zend.assertions = 1` , `assert.exception = 1`) across all four multi-layered scenarios specified in the analytical challenge:

SCENARIO	INITIAL STATE & INPUT PAYLOAD	EXPECTED ENGINE OUTCOME	VERIFIED ASSERTIONS
Scenario 1: FX & Ledger	Account A ($100 USD). Deduct 4,500 INR at 1 USD = 90 INR.	Converts to -$50.00 USD impact; final balance $50.00 USD.	Balance === 50.00; Ledger stores 4500 INR, rate 90.0, impact -50.00.
Scenario 2: VIP Velocity Trap	Account B (VIP, $0 USD). 4 deductions ($100, $100, $100, $50) inside 20s.	First 3 pass (to -$300). 4th breaches >3/60s limit & revokes overdraft.	Throws `InsufficientFundsException` on 4th tx; Balance stays -$300.00.
Scenario 3: Circuit Breaker	Vendor X submits 3 invalid deductions on Accounts C, D, E, then $2 on Account F ($10).	Vendor X trips on 3rd failure; 4th tx blocked despite valid funds.	Throws `CircuitBreakerException` ; Account F stays $10.00; CB log count === 1.
Scenario 4: Batch Rollback	Account G ($100 USD). Atomic batch of [-$40, -$50, -$30] (Total $120 > $100).	3rd deduction fails; entire batch rolls back cleanly in-place.	Throws `InsufficientFundsException` ; Account G === $100.00; Ledger count === 0.

06 Technical Interview Discussion Guide & Q&A

Q1: Why did you choose an in-place Snapshot/Memento method over `clone` for `processBatch()` ?
When client code registers an `Account` object with `SettlementEngine` , both the client script and the engine hold references to the same underlying object ID in PHP's Zend Engine memory. If `processBatch()` overwrites its internal array with a cloned backup object upon failure, the external pointer held by the caller remains mutated by the partially executed transactions. Restoring scalar and array properties in-place via `restoreSnapshot()` ensures zero pointer desynchronization.

Q2: How would this engine transition from in-memory state to a high-concurrency production database?
In production, `processBatch()` maps directly to an RDBMS transaction block (`BEGIN ... COMMIT / ROLLBACK`) using row-level locking (`SELECT ... FOR UPDATE`) on the `Account` record to prevent race conditions. The rolling 60-second velocity window and vendor consecutive failure counters would be backed by Redis sorted sets (`ZCOUNT` by millisecond score) and atomic counters (`INCR` / `DEL`) for sub-millisecond evaluation across distributed PHP-FPM workers.

Q3: Why use float rounding to 4 decimal places in currency conversions?
IEEE 754 floating-point arithmetic can introduce microscopic precision drift during division. Normalizing intermediate FX calculations with `round($value, 4)` prevents floating-point drift during multi-currency settlement while keeping the implementation in pure native PHP without requiring the external `ext-bcmath` extension.

> **Repository Structure & CLI Execution Command**